IoT Project 2026

Only **two** options are available for the implementation of the IoT project:

- **Solo project**
- **Group project**: group composed by 2 students.

No other options are available.

The project **must** be discussed in the **same exam session** in which the oral part of the exam will be given. In the case of group project, the project can be discussed only if at least one of the two members will give the oral part of the exam in the current session.

Project Objective

The goal of the project is to design, implement, and experimentally evaluate an IoT system. The project must demonstrate not only the correct operation of the implemented system, but also the ability to analyze the behavior of the IoT system under different operating conditions, including stress situations.
The use case must be selected by the student(s) among the following application domains:

- **Smart Health:** IoT solutions for monitoring physiological parameters, supporting remote patient care, and enabling data-driven health management and early detection of critical conditions.
- **Demand Side Management (DSM):** IoT solutions for monitoring and controlling energy consumption patterns, including load regulation and reaction to peak-demand conditions.
- **Distributed Energy Systems:** IoT solutions for monitoring and coordinating multiple energy-consuming or energy-producing entities, with mechanisms for load control, prioritization, and resource management.

The selected use case domain must be clearly identifiable in the project design, and the level of detail of its development will be considered as part of the project evaluation.

To explore the details of the use cases, students are encouraged to search for the scientific literature for further inspiration; any external sources used must be properly cited and documented in the final report.

System scheme and structure

The system **must** comprise the following components:

- *A network* of IoT devices, including sensors simulating the collection of data from the physical system/environment and actuators.
- The network must be deployed using real sensors (nrf52840 dongle). In the network a border router must be deployed in order to provide external access (For the sake of simplicity, during the deployment we suggest to simulate the network on Cooja with MQTT and CoAP sensors).
- **[GROUP PROJECT]** The application-layer protocols adopted by the IoT devices **must be** MQTT **and** CoAP. **The choice must be explicitly justified and documented** in the project documentation and **will be considered as part of the evaluation**. Devices using CoAP must implement both CoAP Server functionality, to expose their resources (e.g., sensors or actuators), and CoAP Client functionality for interacting with other devices or services.

- **[SOLO PROJECT]** The application-layer protocol adopted by the IoT devices **may be** MQTT **or** CoAP (or both). **The choice must be explicitly justified and documented** in the project documentation and **will be considered as part of the evaluation**. Devices using CoAP must implement both CoAP Server functionality, to expose their resources (e.g., sensors or actuators), and CoAP Client functionality for interacting with other devices or services.

- A machine learning model that runs on a subset, or all, of the IoT devices. The machine learning model aims at taking **autonomous decisions** on some of the functions of the use case **directly on the IoT device without requiring assistance from external applications**, e.g., by forecasting some metrics and making a decision, by highlighting anomalies and react to them, etc. The machine learning model must be trained accordingly to the procedure presented in class using some open datasets available on the internet, e.g., https://www.kaggle.com/. The colab workbook that is used to train the model must be downloaded and included in the project submission. The workbook must include also a reference to the dataset used.

- The *Cloud Application* collects data from MQTT or CoAP sensors, store them in a MySQL/ InfluxDB database.

- The *Cloud Application* reads information about actuators and sensors from the database and implements a simple *control logic* in order to apply some modifications to one or more actuators based on the data collected from the sensors, e.g. some closed-loop control logic.

- It is required to provide some User Input to implement the User logic for the IoT application. The User input can be implemented as a command line interface.

- **[GROUP PROJECT]** When using CoAP, the CoAP actuators and sensors must register to the Cloud Application, acting as a CoAP Client. This to create a directory of sensors and actuators in the Database. When using MQTT a “registration” topic must be created, to make the registered actuators and sensors available in the specific directory in the Database.

- **[SOLO PROJECT]** The list of actuators and sensors is statically created and exploited by the User Application via a configuration file.

- **Experimental Evaluation and Stress Testing**: Each project must include an experimental evaluation of the implemented IoT system. The evaluation must demonstrate the system behavior under both normal operating conditions and stress situations.
  In particular, for **[GROUP PROJECT]** the system must be tested under the following **two** mandatory stress scenarios:

  - **Network Congestion**: the system must be evaluated under increased traffic load conditions.This can be achieved, for example, by increasing the data generation rate of the nodes, triggering synchronized transmissions from multiple devices, or generating burst traffic events.
    The impact of the stress scenario must measure impact on **latency, packet delivery ratio, number of messages received at the Cloud Application**

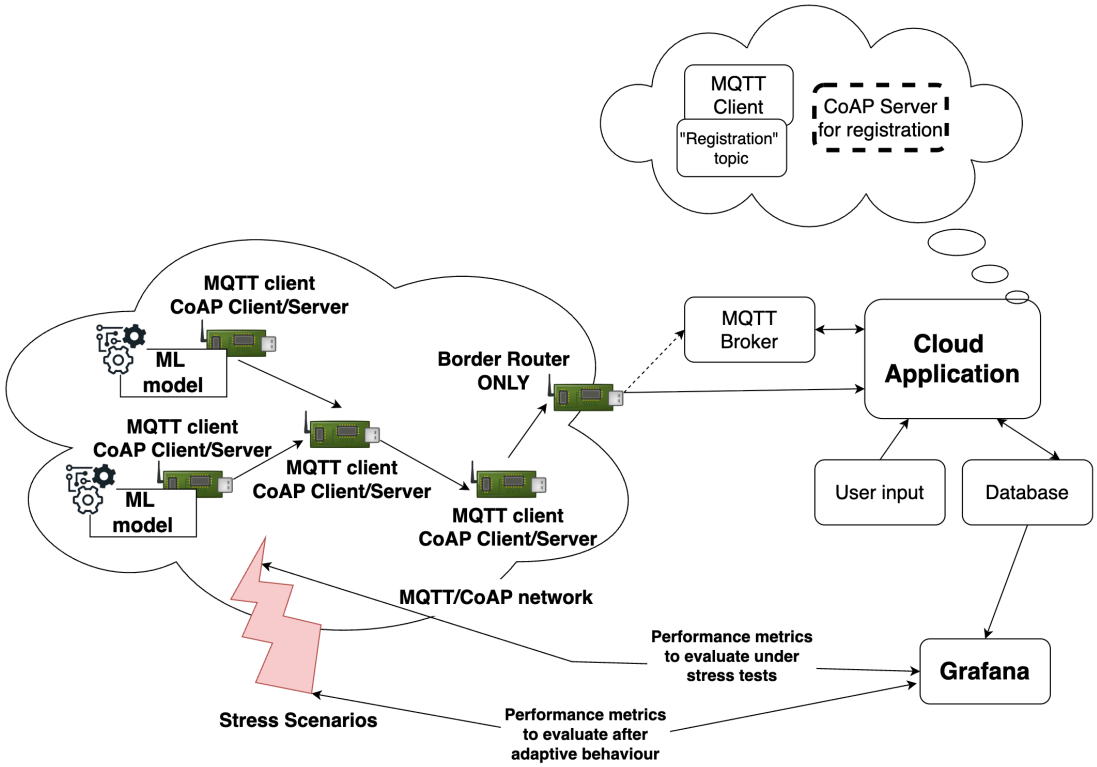
  - **[SOLO PROJECT] Node Failure / Intermittent Behavior Scenario**: The system must be evaluated under conditions where one or more nodes exhibit faulty or unstable behavior. This can be achieved by disabling one or more nodes during execution, introducing intermittent transmissions, delaying or dropping messages from selected devices. The evaluation must show the system ability to detect the failure, the impact on the overall system behavior, the system response to missing or inconsistent data.

- It is required to implement an **adaptive behaviour:** the system must include **one [SOLO PROJECT]** adaptive mechanism that reacts to the stress conditions, **two [GROUP PROJECT]** adaptive mechanisms that react to the stress conditions. Examples of adaptive mechanisms

include: dynamic adjustment of data transmission rate, switching between communication modes (e.g., confirmable/non-confirmable CoAP messages), data aggregation or filtering at the device level, adaptation of control logic based on system conditions. The adaptive mechanism must be clearly described, activated during at least one of the stress scenarios, and evaluated by comparing system performance with and without adaptation.

- A web-based interface deployed using Grafana must be developed in order to show the performance metrics of the stress tests.

System scheme



Guidelines/Requirements

- The application can be implemented using JAVA or Python
- Button **and** LED interactions with sensors **must** be used.
- Data should be encoded in a format that might depend on the specific use-case. The selection of the encoding language should be motivated, and will be part of the evaluation.

Project submission

Projects must be submitted at least **4 days prior to the day of the exam,** e.g., if the exam session is 08/06/2026 the last day for the submission (included) will be 04/06/2026. The project must be submitted using this link: https://forms.gle/17JV9iMam7RWzZZh6. Students must submit the **code (.zip)** of the project and a **document (max 10 pages) describing the implementation and the use-case**.

Project discussion

The project discussion will take place the day of the exam, in person.The discussion, 10 minutes max for SOLO PROJECT, 15 minutes max GROUP PROJECT, will consist on the demo of the execution of the application, showing all the implemented and required features (5/10 minutes), and on 5 minutes questions on the code.